

Task 1 : Write a C program to implement client-server communication using named pipes (FIFOs). The client should send a message to the server, and the server should process the message and return a response to the client.

Codes:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>

#define SERVER_FIFO "/tmp/server_fifo"
#define BUFFER_SIZE 256

int main() {
    char buffer[BUFFER_SIZE];

    // Create named pipe
    mkfifo(SERVER_FIFO, 0666);

    printf("Server started...\n");
    printf("Waiting for clients...\n");

    while (1) {
        int fd = open(SERVER_FIFO, O_RDONLY);

        if (fd == -1) {
            perror("open");
            exit(1);
        }

        while (read(fd, buffer, BUFFER_SIZE - 1) > 0) {
            buffer[strlen(buffer)] = '\0';
            printf("Client message: %s\n", buffer);

            // Process message
            char response[BUFFER_SIZE];

            snprintf(response, BUFFER_SIZE,
                    "Server received: %s\n", buffer);

            // Send response through client's FIFO
            char client_fifo[BUFFER_SIZE];

            // Message format: client_fifo|message
            char *separator = strchr(buffer, '|');

            if (separator != NULL) {
                *separator = '\0';

                strcpy(client_fifo, buffer);
                char *message = separator + 1;

                int response_fd = open(client_fifo, O_WRONLY);

                if (response_fd != -1) {
                    snprintf(response, BUFFER_SIZE,
                            "Server response: processed \"%s\"\n",
                            message);

                    write(response_fd, response, strlen(response));
                    close(response_fd);
                }
            }

            close(fd);
        }

        return 0;
    }
}
```

```

GNU nano 7.2 client.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>

#define SERVER_FIFO "/tmp/server_fifo"
#define BUFFER_SIZE 256

int main() {
    char client_fifo[BUFFER_SIZE];
    char message[BUFFER_SIZE];
    char buffer[BUFFER_SIZE];

    // Unique FIFO for this client
    snprintf(client_fifo, BUFFER_SIZE,
             "/tmp/client_%d", getpid());

    mkfifo(client_fifo, 0666);

    printf("Enter message: ");
    fgets(message, BUFFER_SIZE, stdin);

    // Open server FIFO
    int fd = open(SERVER_FIFO, O_WRONLY);

    if (fd == -1) {
        perror("Server is not running");
        unlink(client_fifo);
        return 1;
    }

    // Send client FIFO name + message
    char request[BUFFER_SIZE];

    snprintf(request, BUFFER_SIZE,
             "%s|%s",
             client_fifo,
             message);

    write(fd, request, strlen(request));

    close(fd);

    // Wait for server response
    int response_fd = open(client_fifo, O_RDONLY);

    if (response_fd == -1) {
        perror("response FIFO");
        unlink(client_fifo);
        return 1;
    }

    int n = read(response_fd, buffer, BUFFER_SIZE - 1);

    if (n > 0) {
        buffer[n] = '\0';
        printf("%s", buffer);
    }

    close(response_fd);

    // Remove client FIFO
    unlink(client_fifo);

    return 0;
}

```

Output:

```

karthik@Karthikeya:~/fifo_client_server$ gcc server.c -o server
gcc client.c -o client
server.c: In function 'main':
server.c:37:48: warning: '%s' directive output may be truncated writing up to 255 bytes into a region of size 239 [-Wformat-truncation=]
   37 |         "Server received: %s\n", buffer);
      |                                     ^
server.c:36:13: note: 'snprintf' output between 19 and 274 bytes into a destination of size 256
   36 |         snprintf(response, BUFFER_SIZE,
      |         ^~~~~~
   37 |         "Server received: %s\n", buffer);
      |         ~~~~~
client.c: In function 'main':
client.c:38:18: warning: '%s' directive output may be truncated writing up to 255 bytes into a region of size between 0 and 255 [-Wformat-truncation=]
   38 |         "%s\n",
      |         ^
   39 |         client_fifo,
      |         ~~~~~
   40 |         message);
      |         ~~~~~
client.c:37:5: note: 'snprintf' output between 2 and 512 bytes into a destination of size 256
   37 |         snprintf(request, BUFFER_SIZE,
      |         ^~~~~~
   38 |         "%s|%s",
      |         ~~~~~
   39 |         client_fifo,
      |         ~~~~~
   40 |         message);
      |         ~~~~~

```

```

karthik@Karthikeya:~$ cd ~/fifo_client_server
./client
Enter message: Hello Server
Server response: processed "Hello Server"
karthik@Karthikeya:~/fifo_client_server$

```

```

karthik@Karthikeya:~$ cd ~/fifo_client_server
./client
Enter message: Client 3 Message
Server response: processed "Client 3 Message"
karthik@Karthikeya:~/fifo_client_server$

```

Task 2:

Codes: Write a C program to demonstrate signal handling using `signal()` or `sigaction()`. Handle signals such as `SIGINT`, `SIGTERM`, and `SIGUSR1` and display appropriate messages when the signals are received.

```

GNU nano 7.2 signals.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>

void signal_handler(int sig)
{
    if (sig == SIGINT)
    {
        printf("\nSIGINT received! (Ctrl+C)\n");
    }
    else if (sig == SIGTERM)
    {
        printf("\nSIGTERM received! Termination signal.\n");
    }
    else if (sig == SIGUSR1)
    {
        printf("\nSIGUSR1 received! User-defined signal.\n");
    }

    fflush(stdout);
}

int main()
{
    // Register signal handlers
    signal(SIGINT, signal_handler);
    signal(SIGTERM, signal_handler);
    signal(SIGUSR1, signal_handler);

    printf("Signal handling program started.\n");
    printf("Process ID (PID): %d\n", getpid());

    printf("\nSignals being handled:\n");
    printf("SIGINT - Ctrl+C\n");
    printf("SIGTERM - kill command\n");
    printf("SIGUSR1 - user-defined signal\n");

    printf("\nWaiting for signals...\n");

    while (1)

```

```

karthik@Karthikeya:~$ kill -SIGUSR1 12345
-bash: kill: (12345) - No such process
karthik@Karthikeya:~$ kill -SIGUSR1 2123
karthik@Karthikeya:~$ kill -SIGTERM 2123
karthik@Karthikeya:~$

```

```

karthik@Karthikeya:~/fifo_client_server$ mkdir -p ~/signal_handling
karthik@Karthikeya:~/fifo_client_server$ cd ~/signal_handling
karthik@Karthikeya:~/signal_handling$ nano signals.c
karthik@Karthikeya:~/signal_handling$ gcc signals.c -o signals
karthik@Karthikeya:~/signal_handling$ ./signals
Signal handling program started.
Process ID (PID): 2123

Signals being handled:
SIGINT - Ctrl+C
SIGTERM - kill command
SIGUSR1 - user-defined signal

Waiting for signals...
^C
SIGINT received! (Ctrl+C)

SIGUSR1 received! User-defined signal.

SIGTERM received! Termination signal.
^C
SIGINT received! (Ctrl+C)
|

```